

## **Task 1: Machine Learning Classification Project**

### **Objective**

Build and compare machine learning classification models using the Iris dataset.

### **Dataset**

Iris Dataset from Scikit-Learn

### **Algorithms Used**

- Logistic Regression
- Random Forest

### **Data Preprocessing**

- Loaded dataset
- Checked missing values
- Performed train-test split
- Conducted exploratory data analysis

### **Evaluation Metrics**

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix

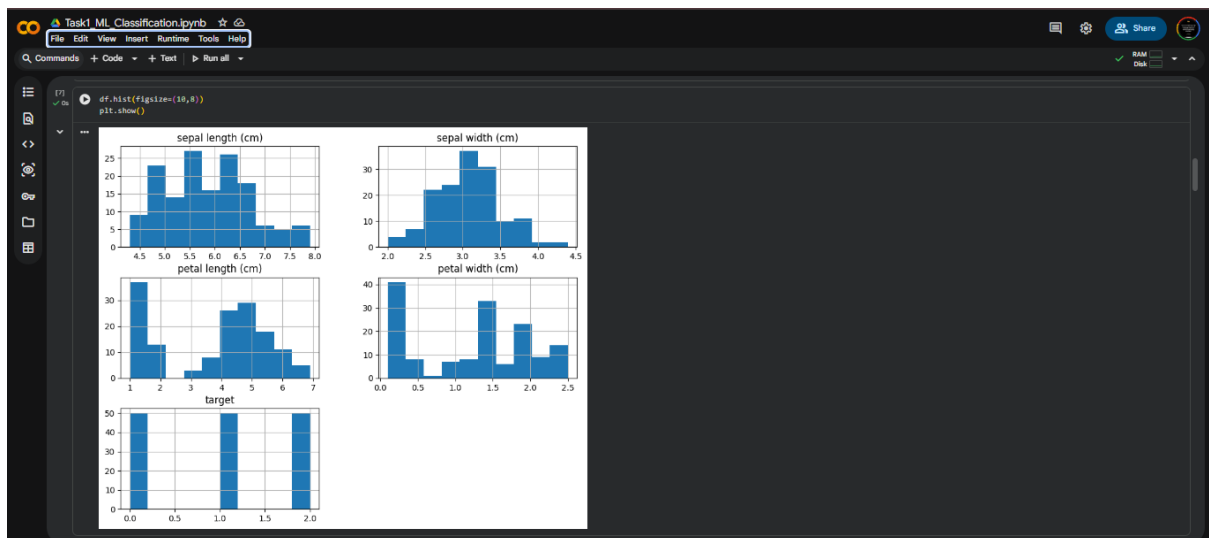
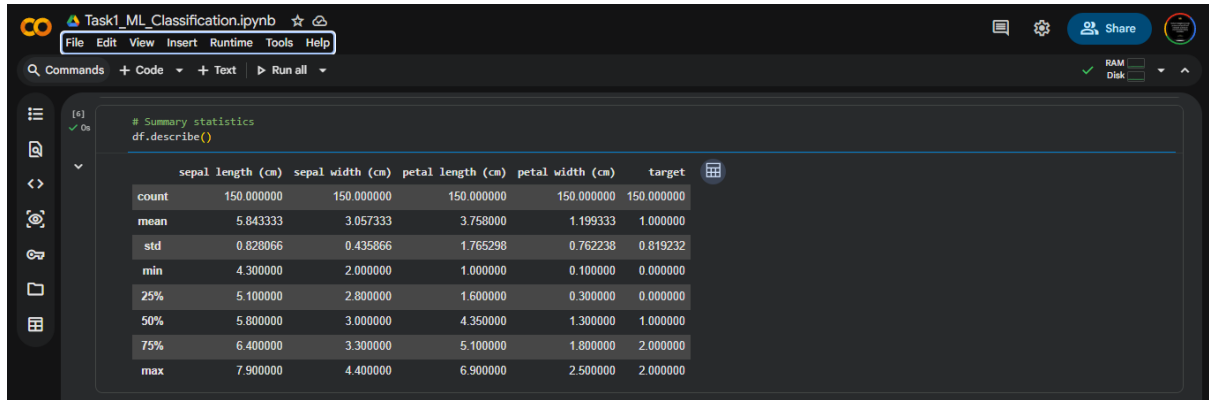
### **Results**

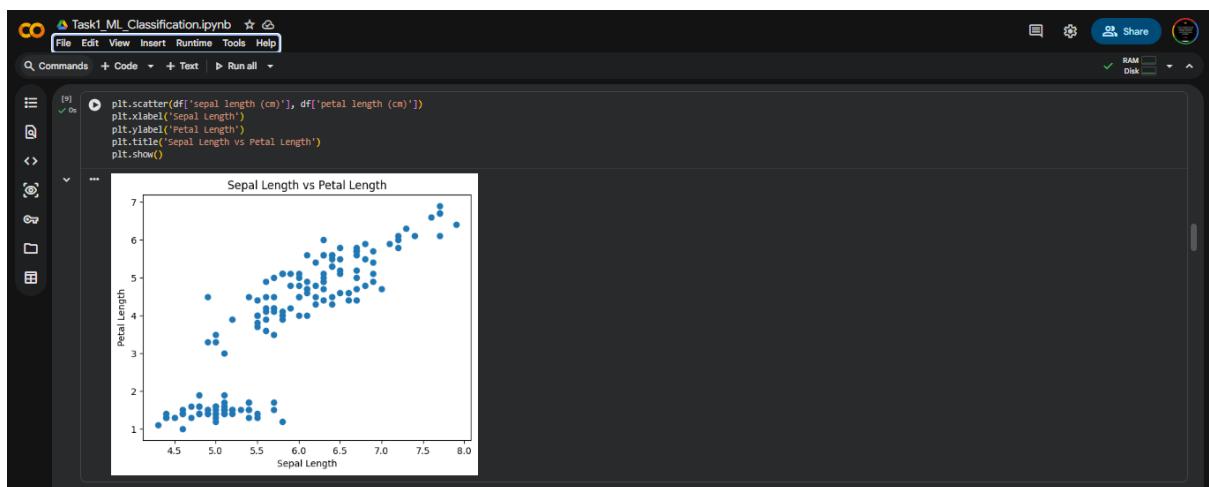
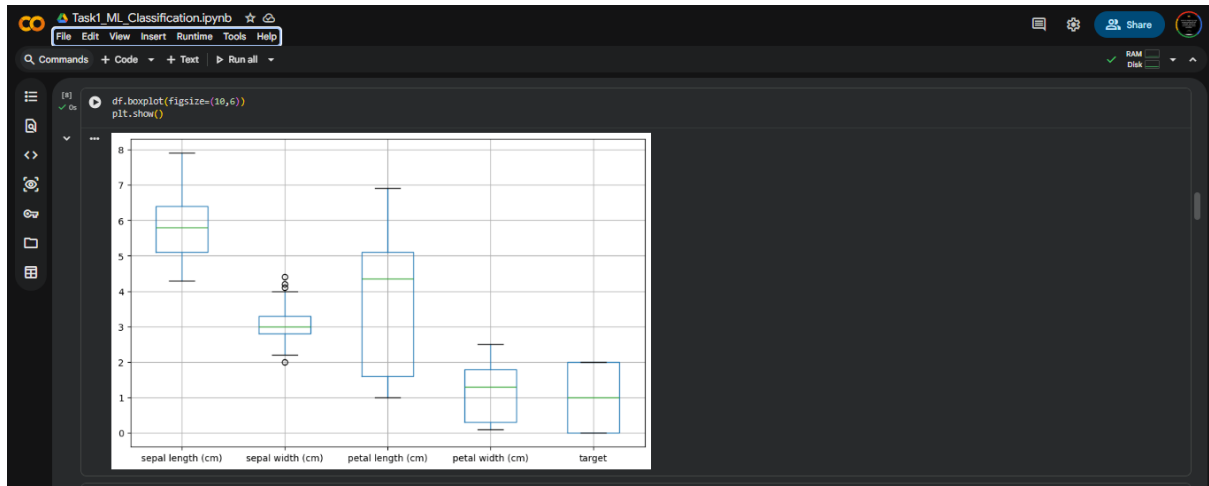
Logistic Regression Accuracy: 100%

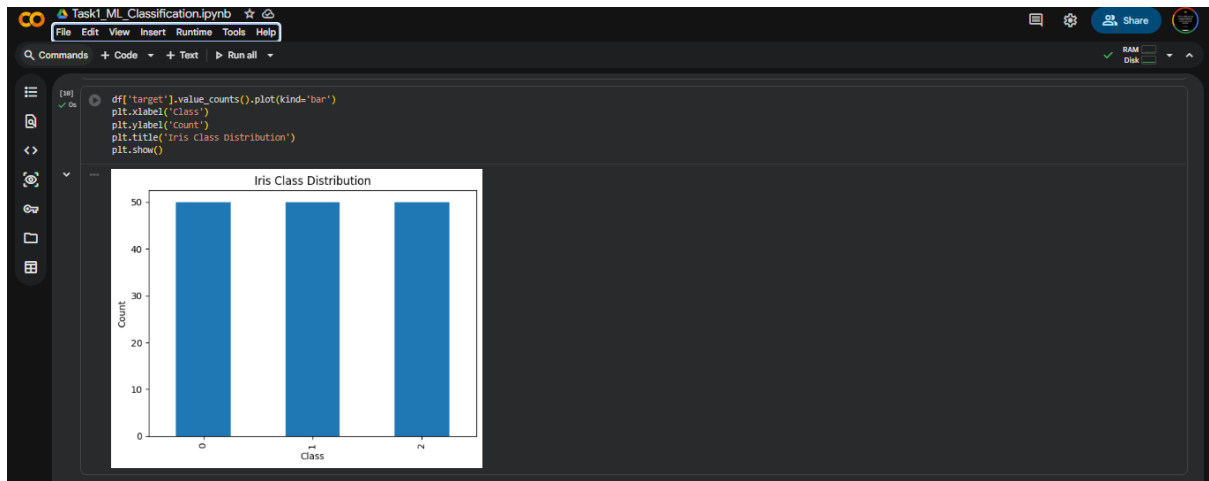
Random Forest Accuracy: 100%

### **Conclusion**

Both models achieved excellent performance on the Iris dataset. Random Forest additionally provided feature importance analysis for better interpretability.







Task1 ML Classification.ipynb

```
[11] from sklearn.model_selection import train_test_split

# Features (X) and Target (y)
x = df.drop('target', axis=1)
y = df['target']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
```

Training data shape: (120, 4)  
Testing data shape: (30, 4)

```
[12] from sklearn.linear_model import LogisticRegression

# Create model
lr_model = LogisticRegression(max_iter=200)

# Train model
lr_model.fit(X_train, y_train)

# Predictions
y_pred_lr = lr_model.predict(X_test)

print("Model trained successfully!")
```

Model trained successfully!

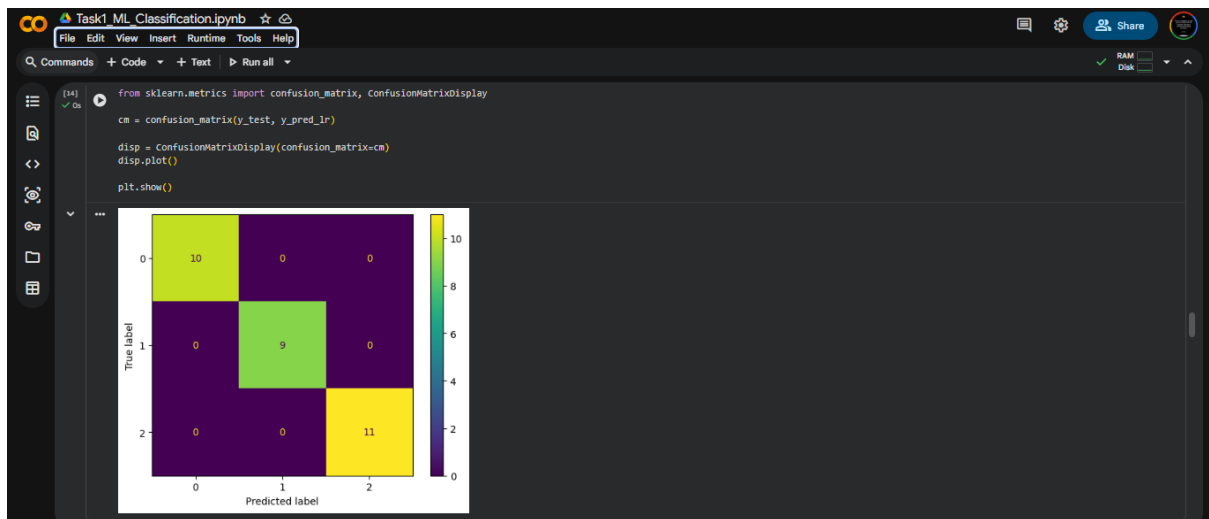
Task1 ML Classification.ipynb

```
[13] from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

accuracy = accuracy_score(y_test, y_pred_lr)
precision = precision_score(y_test, y_pred_lr, average='weighted')
recall = recall_score(y_test, y_pred_lr, average='weighted')
f1 = f1_score(y_test, y_pred_lr, average='weighted')

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
```

Accuracy: 1.0  
Precision: 1.0  
Recall: 1.0  
F1 Score: 1.0



Task1 ML Classification.ipynb

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred_lr))
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 1.00      | 1.00   | 1.00     | 10      |
| 1            | 1.00      | 1.00   | 1.00     | 9       |
| 2            | 1.00      | 1.00   | 1.00     | 11      |
| accuracy     |           |        | 1.00     | 30      |
| macro avg    | 1.00      | 1.00   | 1.00     | 30      |
| weighted avg | 1.00      | 1.00   | 1.00     | 30      |

```
from sklearn.ensemble import RandomForestClassifier

# Create model
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Train model
rf_model.fit(X_train, y_train)

# Predictions
y_pred_rf = rf_model.predict(X_test)

print("Random Forest trained successfully!")
```

Random Forest trained successfully!

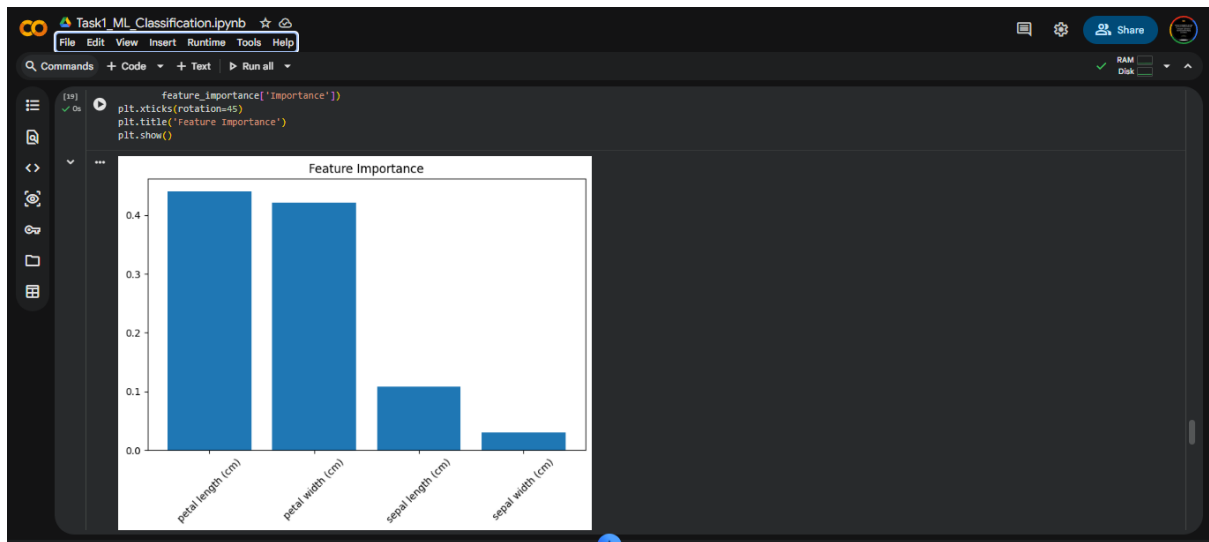
Task1 ML Classification.ipynb

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

accuracy_rf = accuracy_score(y_test, y_pred_rf)
precision_rf = precision_score(y_test, y_pred_rf, average='weighted')
recall_rf = recall_score(y_test, y_pred_rf, average='weighted')
f1_rf = f1_score(y_test, y_pred_rf, average='weighted')

print("Accuracy:", accuracy_rf)
print("Precision:", precision_rf)
print("Recall:", recall_rf)
print("F1 Score:", f1_rf)
```

Accuracy: 1.0  
Precision: 1.0  
Recall: 1.0  
F1 Score: 1.0



Task1\_ML\_Classification.ipynb

```
comparison = pd.DataFrame({
    'Model': ['Logistic Regression', 'Random Forest'],
    'Accuracy': [accuracy, accuracy_rf],
    'Precision': [precision, precision_rf],
    'Recall': [recall, recall_rf],
    'F1 Score': [f1, f1_rf]
})

comparison
```

|   | Model               | Accuracy | Precision | Recall | F1 Score |
|---|---------------------|----------|-----------|--------|----------|
| 0 | Logistic Regression | 1.0      | 1.0       | 1.0    | 1.0      |
| 1 | Random Forest       | 1.0      | 1.0       | 1.0    | 1.0      |

Next steps: [Generate code with comparison](#) [New interactive sheet](#)